Chapitre 0 — Avant tout | Partie C — Déplacer une lettre | leçon 0.30.2 sur 153 | difficulté 1 / 10

0.30.2. Un pas sans attendre : un_pas — doublé, deux positions

Le A (ligne 0) et le B (ligne 8) avancent EN MÊME TEMPS : deux un_pas dans la même image.

1. Ce qu'il faut comprendre

C'est le 0.30 et le 0.30.1 réunis : deux `un_pas` dans la même boucle.

Ensemble, cette fois : `un_pas` n'attend pas, les deux pas se font dans la même image. Les deux lettres avancent côte à côte.

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <poser>` → 0.1.3. La fonction poser() — poser une tuile
> - `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police
> - `#include <un_pas>` → 0.29.4. La fonction un_pas() — un seul pas, sans attendre

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **if (si)** — Teste une condition : si elle est vraie, on fait le bloc qui suit.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **incrémenter (++)** — « x++ » ou « x += 1 » : x augmente de 1. « x-- » : x diminue de 1.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **opérateurs de comparaison** — == égal, != différent, < plus petit, > plus grand, <= plus petit ou égal, >= plus grand ou égal. Le résultat est vrai ou faux.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **poser()** — Met une tuile dans le décor, sur une case de la grille de 20 × 18 (32 × 32 en tout, avec la partie cachée).
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`#include <poser>  // pose une tuile sur une case du fond`	Demande à la console « poser » (ce que c'est : pose une tuile sur une case du fond). Sans cette ligne, le compilateur refuserait de

	l'employer. (Le commentaire le dit : « pose une tuile sur une case du fond ».)
`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`#include <un_pas>  // fait faire un seul pas à une tuile`	Demande à la console « un_pas » (ce que c'est : fait faire un seul pas à une tuile). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « fait faire un seul pas à une tuile ».)
`uint8_t x = 0;  // la colonne du A`	Une variable : une boîte appelée x, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « la colonne du A ».)
`uint8_t y = 0;  // sa ligne`	Une variable : une boîte appelée y, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « sa ligne ».)
`uint8_t xb = 0;  // la colonne du B`	Une variable : une boîte appelée xb, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « la colonne du B ».)
`uint8_t yb = 8;  // sa ligne`	Une variable : une boîte appelée yb, qui retient un nombre de 0 à 255, et qui commence à 8. (Le commentaire le dit : « sa ligne ».)
`uint8_t images = 0;  // le chronomètre`	Une variable : une boîte appelée images, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « le chronomètre ».)
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`poser(x, y, ALPHABET[0]);`	Appel de la fonction poser de la console, avec les arguments x, y, ALPHABET[0]. Ce qu'elle fait : pose une tuile sur la grille du fond — un nom de Tuile, ou un numéro. Un 4e argument est une TAILLE : poser(2, 3, MUR, 5) dessine MUR cinq fois plus grand dans le fond (colonne et ligne en clair).
`poser(xb, yb, ALPHABET[1]);`	Appel de la fonction poser de la console, avec les arguments xb, yb, ALPHABET[1]. Ce qu'elle fait : pose une tuile sur la grille du fond — un nom de Tuile, ou un numéro. Un 4e argument est une TAILLE : poser(2, 3, MUR, 5) dessine MUR cinq fois plus grand dans le fond (colonne et ligne en clair).
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu.
`images++;`	images augmente de 1.
`if (images == 15) {`	Une décision : si images est égal à 15, on fait ce qui est entre les accolades. Sinon, on passe.
`images = 0;`	images prend une nouvelle valeur : 0.
`un_pas(x, y, ALPHABET[0], 1, 0);`	Appel de la fonction un_pas de la console, avec les arguments x, y, ALPHABET[0], 1, 0. Ce qu'elle fait : UN pas, tout de suite, sans attendre : 1 vers la droite ou le bas, -1 vers la gauche ou le haut, 0 immobile. Ne bloque pas : à appeler dans la boucle, pour faire bouger plusieurs choses à la fois. un_pas(x, y, ALPHABET[0], 1, 0); range la nouvelle place dans x et y.

`un_pas(xb, yb, ALPHABET[1], 1, 0);`	Appel de la fonction un_pas de la console, avec les arguments xb, yb, ALPHABET[1], 1, 0. Ce qu'elle fait : UN pas, tout de suite, sans attendre : 1 vers la droite ou le bas, -1 vers la gauche ou le haut, 0 immobile. Ne bloque pas : à appeler dans la boucle, pour faire bouger plusieurs choses à la fois. un_pas(x, y, ALPHABET[0], 1, 0); range la nouvelle place dans x et y.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1115 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        72 o  ← la console, à l'allumage  — la mise en route de la console
  un_pas()        117 o  ← main()  — une fonction fournie par le langage
  main()          104 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EffacerCases  — attend de pouvoir écrire une case
  AdresseCase      10 o  ← effacer(), poser()  — trouve l'adresse d'une case de l'écran
  EffacerCases     12 o  ← effacer()  — efface des cases
  CopierTuiles     18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles          704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette,
DonneesTransfert, Notes
Fonctions du langage appelées :
  poser()   × 3    39 o sur place, dans un_pas(), main()  + AdresseCase (10 o)
  effacer() × 1    15 o sur place, dans un_pas()  + AdresseCase (10 o), EffacerCases (12 o)
  image()   × 1     3 o sur place, dans main()  + AttendreImage (28 o)
Tes fonctions appelées :
  un_pas()  × 2    44 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <un_pas>     // fait faire un seul pas à une tuile
```

2. UNE VARIABLE — X

```
uint8_t x = 0;       // la colonne du A
```

3. UNE VARIABLE — Y

```
uint8_t y = 0;       // sa ligne
```

4. UNE VARIABLE — XB

```
uint8_t xb = 0;       // la colonne du B
```

5. UNE VARIABLE — YB

```
uint8_t yb = 8;       // sa ligne
```

6. UNE VARIABLE — IMAGES

```
uint8_t images = 0;   // le chronomètre
```

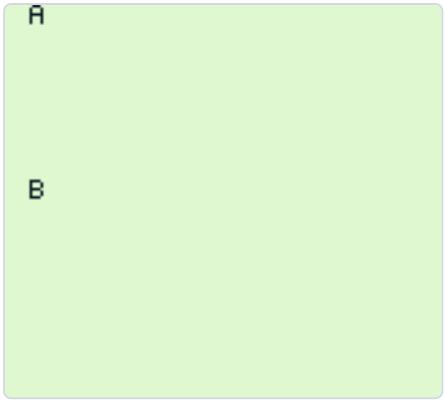
7. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  poser(x, y, ALPHABET[0]);
  poser(xb, yb, ALPHABET[1]);
  while (true) {
    image();
    images++;
    if (images == 15) {
      images = 0;
      un_pas(x, y, ALPHABET[0], 1, 0);
      un_pas(xb, yb, ALPHABET[1], 1, 0);
    }
  }
}
```

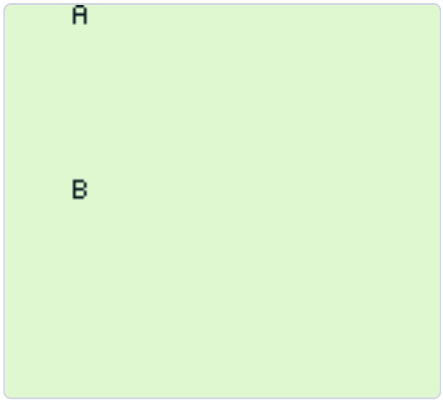
3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

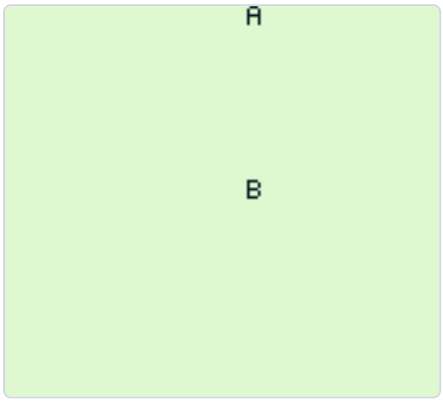
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

> Le A et le B avancent ensemble, l'un sur la ligne 0, l'autre sur la ligne 8.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ ensemble : x et xb sont égaux (7 et 7)

Ensuite : **0.31. Deux lettres à la fois** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++